Summer School on ROS 2 at NTUST: assignments

Author: Dawid Seredyński

June 2026

1 Assignment 1: workspace, package

The following sources can be useful:

- configuring environment,
- creating a workspace,
- creating a package.

1.1 Set up a workspace

Create a workspace in `~/ros2_ws/src`:

```
source /opt/ros/jazzy/setup.bash
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src/
```

1.2 Create a package

Create an `ament_cmake` package; you have to be in the `<workspace>/src` directory:

```
ros2 pkg create --build-type ament_cmake \
    --license Apache-2.0 <package_name>
```

Replace the `<package_name>` with a valid package name. This package will be used to store a robot description of your robot, so a good name is, e.g.: `<YOUR_ROBOT_NAME>_description` (you have to make up the `<YOUR_ROBOT_NAME>`). You can delete the `include` and `src` folders in the new package (they are empty, and they will not be used).

Note: the \ character at the end of a line in the bash command informs the bash that the command is continued in the next line.

1.3 Build

Build the workspace; you have to be in the `<workspace>` directory:

```
cd ~/ros2_ws
clear && colcon build --symlink-install
```

Note: the command prefix "`clear &&`" will clear the terminal and execute the following commands (the build).
Note: `--symlink-install`: non-compiled installed files are symlinked where possible, so you don't have to recompile the package after changing e.g. Python scripts or launch files.

1.4 Set the environment

You can add a command for sourcing `~/ros2_ws/install/setup.bash` in every opened new terminal. Edit the file `~/.bashrc` (it is a hidden file), and add the lines:

```
source ~/ros2_ws/install/setup.bash
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
echo "Sourced to ~/ros2_ws/install/setup.bash"
```

The first command (`source /ros2_ws/install/setup.bash`) will source to your new workspace, while the second (`export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST`) will limit ROS 2 discovery to your machine – ROS 2 will not try to connect to other computers in the lab. If you do so, you have to remember that this particular workspace is sourced in every shell. When working with multiple workspace, you have to be careful not to mix them accidentally – that's why the `echo` command is added in the third line. If there is already a line `source /opt/ros/jazzy/setup.bash` in the `~/.bashrc`, you should add the commands below it.

1.5 Push to Github

Create a new, empty repository on `github.com`, and upload the newly created package. You will find instructions how to make it, on Github – after creating the repository. It is recommended to keep the code developed during the course up-to-date on Github.

Expected result After this assignment, you should have a ROS 2 workspace with one package created, built successfully with `colcon`, sourced in the terminal, and uploaded to a GitHub repository.

2 Assignment 2: the first URDF and launch file

The following sources can be useful:

- building a URDF file,
- creating a launch file
- developing a ROS 2 package

2.1 Add directories and files to the package

Create the following directories in your package: `launch`, `urdf`. Create a URDF file in your package, in folder `urdf`. It may contain one link with one visual, e.g. a cylinder or a box.

Add a new, empty launch file named `description_test.launch.xml` in `launch` directory in your package. The file contents:

```
<?xml version="1.0"?>
<launch>
</launch>
```

Add install command to your `CMakeLists.txt` for installing directories `launch` and `urdf`:

```
install(DIRECTORY launch urdf
        DESTINATION share/${PROJECT_NAME})
```

2.2 Rebuild and run

Build the workspace again and source `install/setup.bash`. If everything is configured properly, you should be able to run (replace `NAME` with your robot's name):

```
ros2 launch NAME_description description_test.launch.xml
```

The launch file is empty, so it does nothing.

2.3 Visualize the robot model

Add the node `robot_state_publisher` from the package `robot_state_publisher` to the launch file. It requires a `robot_description` parameter (the content of a URDF file). You need to read the `robot_description` parameter for `robot_state_publisher` from your URDF file. Define a launch variable for URDF file name; add in the launch file:

```
<let name="urdf_file"
     value="$(find-pkg-share NAME_description)/urdf/NAME.urdf" />
```

To read a single ROS parameter from file use the following substitution (this is not documented in ROS 2 docs, unfortunately); add in the launch file:

```
<param name="robot_description"
       value="$(file-content $(var urdf_file))" type="str" />
```

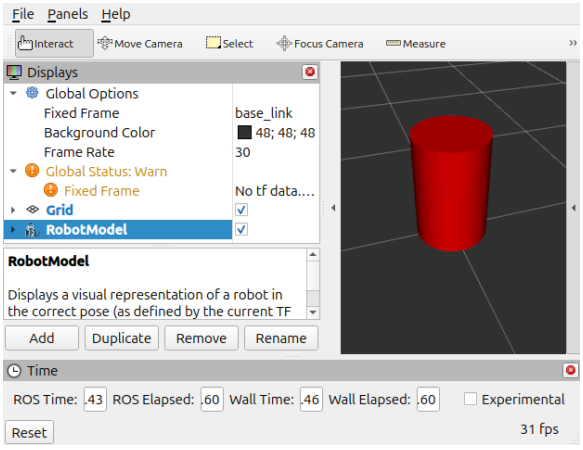
Add a node `rviz2` for visualization (from package `rviz2`). Run the launch file. Change Fixed frame in `rviz2` to `base_link`. Add the `RobotModel` visualization in `rviz2` panel. You should see the robot model. Save the `rviz2` configuration (`File -> Save config as...`) in your package, e.g. as `config.rviz` file in `config` directory (you need to create it).

Remember to add install command for the new directory `config` in the `CMakeLists.txt` and to rebuild. Set the `rviz2` configuration path in the launch file: add `args=...` to the `rviz2` node, e.g.:

```
args="-d $(find-pkg-share NAME_description)/config/config.rviz"
```

2.4 Inspect the running system

Run your launch file. You should see a robot similar to Fig. 1. Use the following commands to inspect



Rysunek 1: A trivial URDF model: one link

topics and nodes:

```
ros2 topic list -v
ros2 topic info -v <topic_name>
ros2 node list
ros2 node info <node_name>
```

Expected result After this assignment, you should have a simple URDF robot model installed in your package, a launch file that starts `robot_state_publisher` and `rviz2`, and an RViz configuration that displays the robot model correctly.

3 Assignment 3: building a robot model

The following sources can be useful:

- building a movable robot model with URDF
- URDF xml syntax

3.1 Create a movable model

Add another link and a revolute joint connecting them. This time, when you run the `rviz2`, the new link is white and it is in the origin. That is because `rviz2` has no information about the pose of the new link – the joint transformation is unknown. There is a `robot_state_publisher` node that publishes on `/tf` topic, but it has no information about joint states of the robot.

Add the node `joint_state_publisher_gui` (from the package `joint_state_publisher_gui`) to your launch file. Run the launch file again. You should see the full robot model and a new window with a slider that allows to control the robot's joint.

Use the following commands to inspect topics and nodes:

```
ros2 topic list -v
ros2 topic info -v <topic_name>
ros2 node list
ros2 node info <node_name>
```

What changed?

3.2 Create a simple manipulator

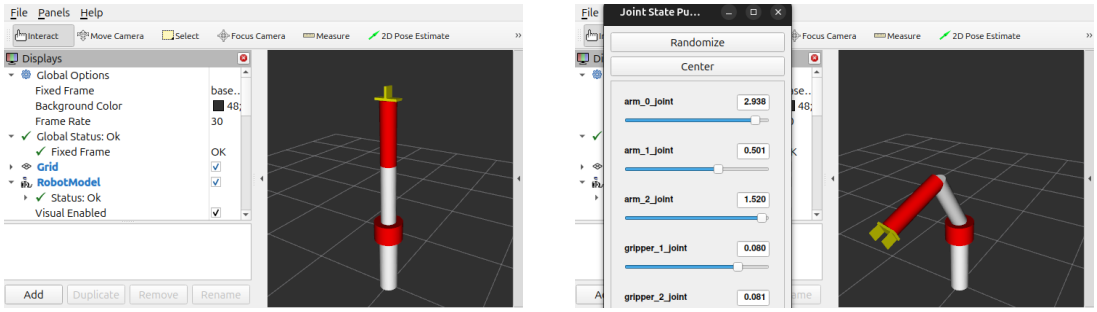
Create a manipulator with three revolute joints (in URDF), and a new launch file that runs rviz2, joint state publisher GUI and robot state publisher.
Note: the position "0" for each joint should be within joint position limits.

3.3 Add a gripper

Add a gripper (jaws on prismatic joints) connected with a fixed joint to the last link of the manipulator. The effect should be similar to Fig. 2.

3.4 Add collision geometry

Add collision geometry to the robot model – the same as visual. You can visualize the collision geometry in `rviz` (in the `RobotModel` visualization). Compare collision geometry to visual geometry in `rviz`.



Rysunek 2: A simple 3 DoF manipulator

3.5 TF visualization

Add `TF` visualization in `rviz2`. You can hide the robot model or change its "Alpha" (the transparency) to observe the TF tree. Move the robot using joint state publisher GUI.

3.6 Static TF

Add a static TF to the TF tree using the following command:

```
ros2 run tf2_ros static_transform_publisher
```

Set arguments in the above command to create a new TF frame attached to the last link of the manipulator with some offset and identity rotation (RPY=[0 0 0]).

Expected result After this assignment, you should have a movable robot model with several joints, a gripper, visual and collision geometry, and a launch file that starts `robot_state_publisher`, `joint_state_publisher_gui`, and `rviz2`. You should be able to move the robot with sliders and observe the TF tree in RViz.

4 Assignment 4: motion generation in a simple kinematic simulation

- creating a package (Python)
- writing a simple publisher and subscriber in Python
- documentation for the JointState message

4.1 Create a Python Package

Create a new Python package with a Python node, as presented in the source URL. It will contain a few scripts used in next assignments. You can use the following command:

```
ros2 pkg create --build-type ament_python --license Apache-2.0 \
--node-name <node_name> <package_name>
```

4.2 Create a Python node for motion generation

Write a Python node that generates a smooth trajectory that is visualized in `rviz2`. This is not a full simulation yet, as there is no physics (collision detection, contact forces, dynamics, constraints, etc.).

Write a node that publishes on the topic `/joint_states`, so the robot can move in `rviz2`. The node should publish periodically, e.g. every 50 milliseconds (use a timer). You also need to create a new launch file (you can copy the previously used launch file) and:

- remove the joint state publisher GUI node, because the new node will publish /joint_states,
- add the node.

You can generate the joint trajectory by interpolating robot's position in the joint space between a number of points in the joint space.

- the generated trajectory should be smooth (position only, velocity can be not continuous),
- define a few points (e.g. 5) in the joint space (trajectory nodes),
- smoothly interpolate current position between trajectory nodes,
- when the last point is reached, the algorithm can interpolate to the first point, or go backwards.

Expected result After this assignment, you should have a Python ROS 2 node that publishes smooth joint trajectories on the `/joint_states` topic. The robot should move automatically in RViz without using `joint_state_publisher_gui`.

5 Assignment 5: a robot model for simulation and planning

The following sources can be useful:

- URDF xml specification for link and inertia parameters,
- explanation of inertial parameters, see the "List of 3D inertia tensors",
- a tutorial for MoveIt Setup Assistant.

5.1 Complete the robot description

To run a full simulation, the robot model must be properly configured. Now it is lacking inertial parameters (e.g. mass, moment of inertia). Add inertial parameters to the robot model in URDF.

Assume a mass for each link that is realistic and consistent with link size, e.g. 1 kg for a link of length 0.5 m. Set the same center of mass ("origin" in "inertial" xml node) as origin of visual. Calculate moment of inertia tensor for a solid of the same shape as visual (use formulas in "List of 3D inertia tensors").
Note: the example inertia tensor in URDF xml link specification is very unrealistic and may cause strange behavior in simulation. The `<inertial>` element must be properly defined for all links.

5.2 Generate MoveIt configuration

Run `MoveIt Setup Assistant` and:

- create a new MoveIt configuration package for your robot,
- create a self-collision matrix,

- define planning groups for: arm, gripper,
- define end-effector,
- add `ros2_control`,
- add `JointTrajectoryController` for both planning groups (arm, gripper),
- add `MoveItController` for both planning groups,
- fill the author information,
- generate the package; a good name for a package with MoveIt configuration is: `<your_robot_name>_moveit_config`.

Examine the output files in the MoveIt configuration package and check:

- what interfaces are defined in `<robot_name>.ros2_control.xacro` file?
- what controllers are defined in `ros2_controllers.yaml` file?

Note: there are no Gazebo Sim–related tags in the robot description generated with MoveIt Setup Assistant.

5.3 Add Gazebo Sim support

To enable support in Gazebo Sim:

- add in `<robot_name>.urdf.xacro` file:

```
<gazebo>
  <plugin filename="libgz_ros2_control-system.so"
          name="gz_ros2_control::GazeboSimROS2ControlPlugin">
    <parameters>$(find NAME_moveit_config)/config/ros2_controllers.yaml</parameters>
    <controller_manager_name>controller_manager</controller_manager_name>
  </plugin>
</gazebo>
```

  Note: set a proper package name instead of `NAME_moveit_config`.

- change in `<robot_name>.ros2_control.xacro` file:
  - from:
    `<plugin>mock_components/GenericSystem</plugin>`
  - to:
    `<plugin>gz_ros2_control/GazeboSimSystem</plugin>`

5.4 Generate the complete URDF

Now you have to create a complete URDF file with your robot model. MoveIt Setup Assistant generated additional wrappers for your robot URDF using `xacro`, so the model is much more complex now. The main `xacro` file is `<robot_name>.urdf.xacro` in the MoveIt configuration package. The `xacro` file includes several other files, including the original URDF with your robot description.

To generate a new, "final" URDF file, run the following command:

```
xacro path_to_urdf_xacro_input > path_to_urdf_output
```

You can check the generated "final" URDF file: what has changed compared to the original URDF?

Note: the process of creating a final URDF from the original URDF and xacro wrappers generated by MoveIt Setup Assistant is usually done automatically at startup of the system. This process is defined in the "bringup" (or "spawner", if in simulation) launch file.

Expected result After this assignment, your robot model should contain inertial parameters for all links and a generated MoveIt configuration package. You should understand the generated planning groups, controllers, ROS 2 control interfaces, and the additional xacro wrappers created by MoveIt Setup Assistant.

6 Assignment 6: running the robot in Gazebo Sim

The following sources can be useful:

- launching Gazebo Sim from ROS 2,
- an example launch file for running Gazebo Sim,
- configuration of Gazebo Sim ROS 2 bridge,
- spawning a robot in Gazebo Sim,

6.1 Create a new package

Create a new package `<robot_name>_gazebo` (`ament_cmake`) with:

- launch files,
- configuration files (e.g. for gz bridge, rviz),
- worlds.

6.2 Create a launch file for the simulator

Note: when running Gazebo Client under VirtualBox, you may encounter rendering problems. To avoid that, Gazebo Client must be run with a legacy rendering engine (option `--render-engine ogre`).
Note: when running Gazebo Sim under VirtualBox, spawning a robot in the launch file that runs Gazebo server may not work properly. To avoid that, use separate launch files for running Gazebo Sim and for spawning a robot (and all its controllers). If you want to use a single launch file anyway, you have to implement a waiting mechanism, such that spawning a robot waits for Gazebo Sim bringup (the first message on the `/clock` topic).

Create a launch file `start_simulator.launch.xml` that runs Gazebo Sim with a specified world file. Example launch file URL is listed in the sources for the assignment.

This launch file should run `gz_server`, `gz_client` and `ros_gz_bridge`. To run Gazebo Client (GUI, `gz_client`) you can use the following xml code:

```
<!-- Client / GUI only -->
<include file="$(find-pkg-share ros_gz_sim)/launch/gz_sim.launch.py">
  <arg name="gz_args" value="-g"/>
  <arg name="on_exit_shutdown" value="True"/>
</include>
```

If running under VirtualBox, use the legacy rendering engine, so change the line:

```
  <arg name="gz_args" value="-g"/>
```

to:

```
        <arg name="gz_args" value="-g --render-engine ogre"/>
```

Configure the `ros_gz_bridge` in the launch file: create a separate `yaml` file with bridge parameters and load it in the launch file. At least the `/clock` topic should be bridged.

Add a new empty world and load it from launch file.

6.3 Create a launch file for spawning the robot

Create a launch file `spawn_robot.launch.xml` that spawns the robot into Gazebo Sim. Gazebo Sim uses SDF file format as description of models (or robots). However, Gazebo Sim can also read robot model directly from URDF, so you don't have to convert from URDF to SDF.

Add the `robot_state_publisher` node in the launch file. The `robot_state_publisher` node requires a `robot_description` parameter. It may look similar as in the Assignment 2, but this time the robot model is extended with `xacro` wrappers generated by MoveIt Setup Assistant. You need to set the `robot_description` parameter for `robot_state_publisher` using the `.urdf.xacro` file and `xacro` parser. To build a final URDF parameter by parsing the `.urdf.xacro` file, you can use the following code:

```
<param
  name="robot_description"
  value="$(command 'xacro $(find-pkg-share NAME_moveit_config)/config/NAME.urdf.xacro')"
  type="str" />
```

You should also add the following nodes to the launch file:

- robot model spawner for Gazebo Sim: `<gz_spawn_model>` (see the "spawning a robot in Gazebo Sim" source); read the robot model using topic `/robot_description` (set the `topic="/robot_description"`) for `<gz_spawn_model>`); you can do so, because `robot_state_publisher` publishes the loaded URDF on this topic,
- a spawner for ROS 2 controller `joint_state_broadcaster` (it requires the `robot_state_publisher` to get `robot_description` to know what joints are available),
- `rviz2` with configuration loaded from the package.

Remember to set `use_sim_time` for `rviz2` and other nodes.

6.4 Launch the system

Launch the Gazebo Sim, wait until it is fully initialized, and then launch `spawn_robot.launch.xml`. Finally, you should:

- see the following log:

```
[gzserver-1] [INFO] [1780836807.846003734] [controller_manager]: Successfully
switched controllers!
[spawner-5] [INFO] [1780836807.858180918] [spawner_joint_state_broadcaster]:
Configured and activated joint_state_broadcaster
```

- be able to read `/joint_states` topic,
- see the whole robot in `rviz2` and in Gazebo Client.

Note: if the robot flips over, its base is not fixed. You should add in the URDF a link named "world" (no visual or inertial is needed) and a fixed joint connecting "world" and "base_link" in the original URDF. Regenerate the "final" URDF and SDF (you don't have to run the MoveIt Setup Assistant again).

The robot is loaded in the simulator, its joint states are published by a ROS 2 controller (`joint_state_broadcaster`) running in the simulator, TFs are calculated by `robot_state_publisher`, so `rviz2` and all other nodes know where the links are.

Still, the other controllers (for the arm and the gripper) are not loaded, and MoveIt is not running.

Expected result After this assignment, you should be able to start Gazebo Sim, spawn your robot into the simulation, activate the `joint_state_broadcaster`, read the `/joint_states` topic, and see the robot both in Gazebo Client and in RViz.

7 Assignment 7: controllers and MoveIt

In this assignment you will load ros2_control controllers and run them in Gazebo Sim.

Look into `ros2_controllers.yaml` file. What are the names of controllers? Add a spawner for each controller in the launch file. You can use the following code:

```
<node pkg="controller_manager"
      exec="spawner"
      output="screen"
      args="CONTROLLER_NAME -c /controller_manager" />
```

Set the proper name in place of `CONTOLLER_NAME`.

Due to a bug in MoveIt Setup Assistant, you need to add "action namespace" parameter for all controllers in moveit_controllers.yaml:

```
action\_ns: follow\_joint\_trajectory
```

e.g.:

```
  CONTROLLER_NAME:
    type: FollowJointTrajectory
    action_ns: follow_joint_trajectory
    joints:
      ...
```

In the launch file, include move_group.launch.py from MoveIt configuration package. Run the launch file.

If there is an error:

```
[move_group-9] [ERROR] [move_group.moveit.moveit.core.time_optimal_trajectory_
generation]: No acceleration limit was defined for joint arm_0_joint! You have
to define acceleration limits in the URDF or joint_limits.yaml
```

then fix it, as the error message suggests.

If there is an error:

```
[rviz2-10] [WARN] [1780838151.257620804] [rviz2.moveit.ros.planning_scene_monitor]:
Maybe failed to update robot state, time diff: 1780838135.251s
```

then the `move_group` node does not use simulation time. Unfortunately, automatically generated `move_group` launch file does not support simulation time. You need to rewrite the launch file. You can use the code example on how to rewrite `generate_move_group_launch` function.

If the system has launched successfully, add the `MotionPlanning` panel in `rviz2` and try to move the robot. Select OMPL as the Planning Library.

What actions / topics / services are available? Examine `rqt_graph`.

Expected result After this assignment, the arm and gripper controllers should be loaded and active in Gazebo Sim, MoveIt should be running, and the robot should be controllable from the `MotionPlanning` panel in RViz. You should also be able to inspect the system using topics, services, actions, and `rqt_graph`.

8 Assignment 8: remote control with rviz2 and MoveIt

Add a small object to the world. The size of the object should allow grasping it by the gripper.

You can use a package with models (git clone). You can also add your own model.

To make the models in the package available in your world, you should expand environment variable `GZ_SIM_RESOURCE_PATH`, preferably in the launch file that runs Gazebo Sim:

```
<set_env name="GZ_SIM_RESOURCE_PATH"
  value="$(find-pkg-share wut_gazebo_models)/models:$(env GZ_SIM_RESOURCE_PATH '')" />
```

You should also add dependency in the `package.xml` to the package with models.

Use Resource Spawner in Gazebo Client to add models to the world. Try to grasp the object using manual control with the `MotionPlanning` panel in `rviz2`.

Expected result After this assignment, you should have a Gazebo world with a graspable object, available model resources, and a robot that can be manually commanded with MoveIt from RViz to approach and grasp the object in simulation.